SENA · TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE SOFTWARE

Documento de evidencias

Frontend en React del proyecto QR Empresarial

PROYECTO	QR Empresarial — perfiles empresariales con código QR dinámico
APRENDIZ	Jean Carlos Orozco Hernández
FICHA	3336250
GUÍAS	Frontend N.º 1 y N.º 2, más los ocho módulos del proyecto
REPOSITORIO	github.com/JECAOR/qr-empresarial
VERSIÓN	v3.5.0
FECHA	31 de octubre de 2026

Todas las capturas de este documento se tomaron contra el servidor real, con los datos de `database/seed.sql`, y se regeneran con un guion versionado.

1. Conceptos fundamentales utilizados

El frontend es una aplicación de una sola página construida con **React 19** y **Vite 8**, que consume la API REST del propio proyecto. No accede nunca a la base de datos: pide y envía datos por HTTP, y es el backend quien decide qué se puede hacer.

Componentes, props y composición

Cada pieza de interfaz es una función que devuelve JSX. Lo que cambia entre usos llega por **props**: `TarjetaContenido` recibe el título, el detalle y el tipo de bloque, y decide con una condición si se dibuja como enlace o como texto. Así la misma pieza sirve para los seis bloques del perfil sin repetir código.

Estado y efectos

`useState` guarda lo que cambia mientras la pantalla está abierta y avisa a React de que debe volver a dibujar. `useEffect` sirve para lo que ocurre fuera del dibujado —pedirle los datos al servidor— y su lista de dependencias decide cuándo se repite: con `[dias]`, solo al cambiar el periodo de las estadísticas.

Cada efecto que pide datos devuelve una función de limpieza. Sin ella, si se consulta un negocio y antes de que llegue la respuesta se consulta otro, la primera podría pisar a la segunda.

> **Dos peticiones en desarrollo no son un error.** `StrictMode` ejecuta cada efecto dos veces a propósito, para que salte a la vista un efecto que no sabe limpiarse. En producción se ejecuta una sola vez.

Contexto

La sesión —quién eres, tu rol, entrar y salir— vive en un **contexto** (`AuthContext`) en lugar de pasarse por props de pantalla en pantalla. Al abrir la aplicación no se confía en lo que hay guardado en el navegador: se le pregunta al servidor con `GET /usuarios/me`, de modo que un token caducado o revocado se detecta enseguida.

Enrutado y rutas protegidas

React Router decide qué pantalla se dibuja según la dirección. `RutaProtegida` exige sesión y, cuando hace falta, un rol.

> **El guardia no es seguridad.** Quien escriba la dirección a mano o llame a la API con `curl` se lo salta entero. La seguridad la aplica Express en cada endpoint: una cuenta de empresa recibe 403 aunque consiga pintar la pantalla. Lo que hace el guardia es no ofrecer puertas que van a dar error.

Hooks propios

Un **hook propio** es una función que agrupa estado y efectos para poder reutilizarlos. Hay dos. `useTitulo(titulo)` pone el título de la pestaña, y lo llama cada pantalla: bajo React hay un solo `index.html`, así que sin esto las trece dirían lo mismo. `useTemaPropio()` pide el perfil del negocio y aplica su color de marca; lo llama la barra de navegación, que envuelve a todas las pantallas del panel, de modo que el perfil se pide una vez por sesión y no en cada pantalla.

> **Por qué un hook y no un componente.** Ninguno de los dos dibuja nada: uno escribe en `document.title` y el otro en el `<html>`. Un componente que no devuelve interfaz es un componente de mentira; un hook dice exactamente lo que es, y se puede llamar desde donde conviene.

Separación en capas

Ninguna pantalla llama a `fetch`. Todas pasan por un servicio, y todos los servicios por un único `apiClient` que pone la dirección base, el tipo de contenido y el token, y convierte un error del servidor en una excepción con su mensaje. Si mañana cambia la forma de autenticarse, se toca un archivo.

2. Componentes implementados y su justificación

2.1 Estructura del proyecto

```
src/
  context/AuthContext.jsx        La sesión: quién eres, entrar, salir y refrescar
  components/
    Layout/                      Barra de navegación común
    RutaProtegida/               Exige sesión y, si hace falta, un rol
    Formulario/CampoTexto        Etiqueta, campo y ayuda, en una sola pieza
    FichaNegocio/                Datos del negocio; omite los campos vacíos
    TarjetaContenido/            Un bloque: enlace o texto, según su tipo
    contenido/ListaBloques       Los bloques con su orden y sus acciones
    contenido/FormularioBloque   Alta y edición de un bloque
    usuarios/TablaUsuarios       El listado; en móvil se vuelve fichas
    usuarios/FormularioUsuario   Alta y edición de una cuenta
    perfil/SelectorTema          El color de marca
    estadisticas/GraficaActividad La gráfica diaria, en SVG
    admin/DetalleCuenta          La actividad reciente de una cuenta
  pages/                         Trece pantallas
  services/                      Once servicios, uno por módulo de la API
  hooks/
    useTemaPropio                El color de marca del negocio con sesión
    useTitulo                    El título de la pestaña de cada pantalla
  styles/
    global.css                   Variables de color y los tres temas
    tema.js                      Aplica el color de marca al <html>
  pruebas/                       Lo que montan las pruebas de jsdom

public/                          Lo que se publica tal cual
  manifest.json                  Para que se pueda instalar
  service-worker.js              Red primero, caché de respaldo
  robots.txt, sitemap.xml        Qué puede indexar un buscador

Cada archivo .test.jsx vive al lado de lo que prueba, no en una
carpeta aparte: asi se ve de un vistazo que tiene pruebas y que no.
```

2.2 Por qué cada decisión

COMPONENTE	JUSTIFICACIÓN
apiClient	Única puerta de salida hacia la API. El ejemplo de la guía comprueba body.success; esta API no tiene ese campo, y fiarse de él haría que todo pareciera correcto. Manda el código HTTP.
AuthContext	La sesión la necesitan la barra, el guardia y cuatro pantallas. Pasarla por props obligaría a atravesar componentes que no la usan.
RutaProtegida	Un solo sitio donde se decide quién pasa. Recuerda a dónde iba el usuario para devolverlo ahí después de entrar.
CampoTexto	Las cuatro pantallas de cuenta repetían el mismo bloque de etiqueta, campo y texto de ayuda.
GraficaActividad	SVG dibujado a mano, sin librería: son dos series de entre siete y noventa puntos, y traer chart.js con react-chartjs-2 costaría 70 KB comprimidos —la aplicación pasaría de 98 a 169, un 72 % más—, medido instalándolos y compilando. El <title> de cada punto da la información al pasar el cursor, que es cosa del navegador.
DetalleCuenta	Va en un <dialog> para que el navegador se encargue del foco, de Escape y de bloquear lo de detrás.
TablaUsuarios	Bajo 780 px cada fila se convierte en una ficha con el nombre de la columna delante de su valor. Una tabla de siete columnas en un móvil obliga a desplazar la pantalla a lo ancho para alcanzar los botones.

2.3 Dependencias

Tres en producción y trece de desarrollo, ocho de ellas solo para las pruebas. **Ninguna librería de interfaz**: los estilos, la gráfica y los diálogos son propios.

PAQUETE	VERSIÓN	PARA QUÉ
react / react-dom	19.3	La biblioteca y su puente con el navegador
react-router-dom	7.18	Enrutado y rutas protegidas
vite	8.3	Servidor de desarrollo y empaquetado
@vitejs/plugin-react	6.1	Compilación de JSX y recarga en caliente
oxlint	1.88	Análisis estático
vitest	3.2	El motor de las pruebas; usa la misma configuración que Vite
@testing-library/react	16.3	Monta un componente y lo consulta como lo haría una persona
jsdom	30.1	El navegador de mentira donde se dibujan
@playwright/test	1.64	Chromium de verdad, para las pruebas de extremo a extremo
@axe-core/playwright	4.14	Auditoría de accesibilidad sobre la pantalla ya dibujada

Las de desarrollo no entran en lo que se descarga el visitante. Resultado: **103 KB comprimidos** para trece pantallas —98 de JavaScript y 5 de estilos—, medidos con `npm run build`.

3. Evidencia de integración con la API

El frontend consume **39 de los 40 endpoints** de la API REST del proyecto. El que falta es `GET /api/health`, que solo comprueba que el servidor responde.

`POST /api/publico/:slug/eventos` —el que cuenta escaneos, visitas y clics— lo llama **solo** la página del código QR, `/p/:slug`. El panel del dueño no, o inflaría sus propias estadísticas cada vez que abriera su perfil. La diferencia no se ve en pantalla: equivocarse ahí no rompe nada, solo deja las cifras mintiendo, y por eso tiene pruebas propias.

3.1 Endpoints por módulo

MÓDULO	ENDPOINTS	PANTALLA
Autenticación	/auth/login, /logout, /registro, /recuperar-contrasena, /restablecer/:token, /restablecer-contrasena	Entrar, Registro, Recuperar, Restablecer
Cuenta propia	/usuarios/me (GET, PUT, DELETE), /me/contrasena, /me/preferencias (GET, PUT)	Configuración
Usuarios	/usuarios (GET, POST), /:id (GET, PUT, DELETE), /:id/estado	Gestión de usuarios
Perfil	/perfiles/me (GET, PUT), /me/tema	Mi negocio
Contenido	/contenido (GET, POST), /categorias, /orden, /:id (PUT, DELETE), /:id/estado	Contenido
Código QR	/qr (GET, POST), /estado, /imagen	Código QR
Vista pública	/publico/:slug, /:slug/eventos	Perfil público, página del QR
Estadísticas	/estadisticas	Actividad
Administración	/admin/resumen, /admin/usuarios, /:id, /reportes/usuarios	Administración

3.2 Lo que hubo que adaptar del ejemplo de las guías

Las guías usan SICE como proyecto orientador y advierten que cada equipo debe comprobar el contrato real de su API. Al hacerlo aparecieron ocho diferencias, todas comprobadas contra el servidor con `curl` antes de escribir una sola pantalla:

EL EJEMPLO ASUME	ESTA API RESPONDE
Campo success en el cuerpo	No existe: manda el código HTTP
data.user y expiresIn	data.usuario, sin expiresIn
role como objeto {id, name}	rol como cadena
{email, password} al entrar	{correo, contrasena}
Cuatro roles	Dos: empresa y administrador
/users/:id/status	/usuarios/:id/estado
Estado como 1 o 0	Como palabra: activo o restringido
El CRUD abierto a varios roles	Solo administrador

> Copiar el ejemplo tal cual habría dado `contenido.map is not a function` y una pantalla sin rol con la que no se puede proteger nada. La guía lista ese error entre los frecuentes.

3.3 Comprobación de las peticiones

La colección de Postman del proyecto trae dos carpetas dedicadas a este frontend, con **46 peticiones y 44 pruebas automáticas** que recorren los mismos endpoints en el mismo orden que las pantallas, incluidos los casos negativos —400, 401, 403, 404 y 409— y el contraste de permisos entre un administrador y una empresa.

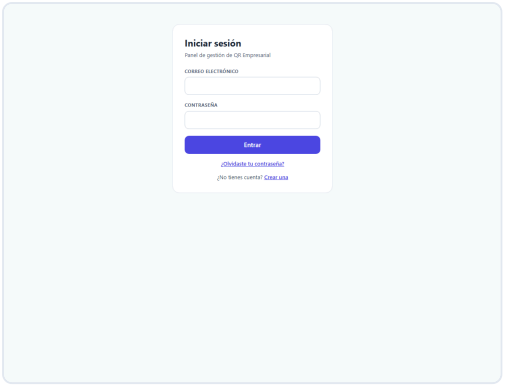
```
Guia React — Frontend N.1    3 peticiones   13 pruebas
Guia React — Frontend N.2   15 peticiones   31 pruebas
```

Crean su propia cuenta de prueba y la eliminan al terminar, de modo que dejan la base como la encontraron.

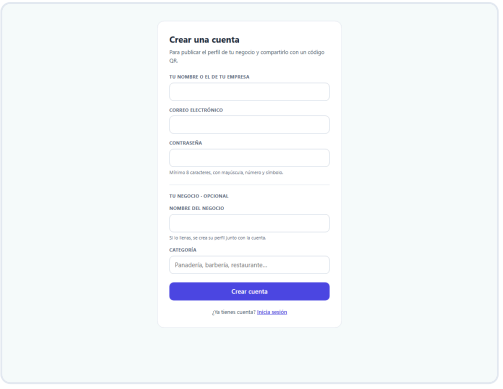
4. Capturas de la interfaz

Las dieciséis capturas siguientes las genera `frontend-react/evidencias/generar-capturas.js`, que abre un navegador, entra con las cuentas de demostración y recorre la aplicación **contra el servidor real**. Lo que se ve sale de MySQL: una pantalla dibujada con datos inventados no demuestra nada.

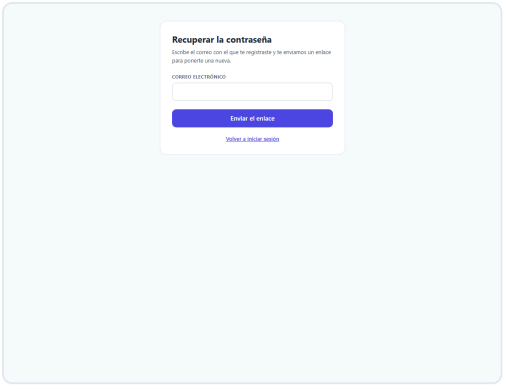
4.1 Acceso y cuenta



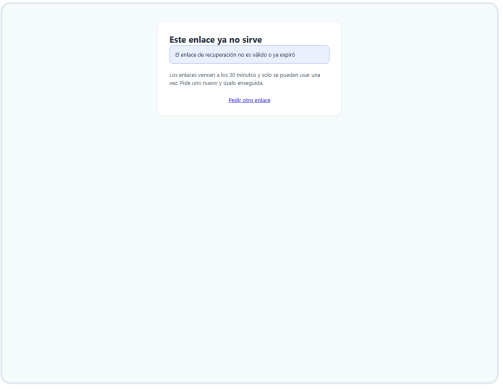
01 · Iniciar sesión. Guarda el token en `sessionStorage`: la sesión muere al cerrar la pestaña.



02 · Registro. El negocio es opcional; si se llena, la cuenta y su perfil se crean en una misma transacción.

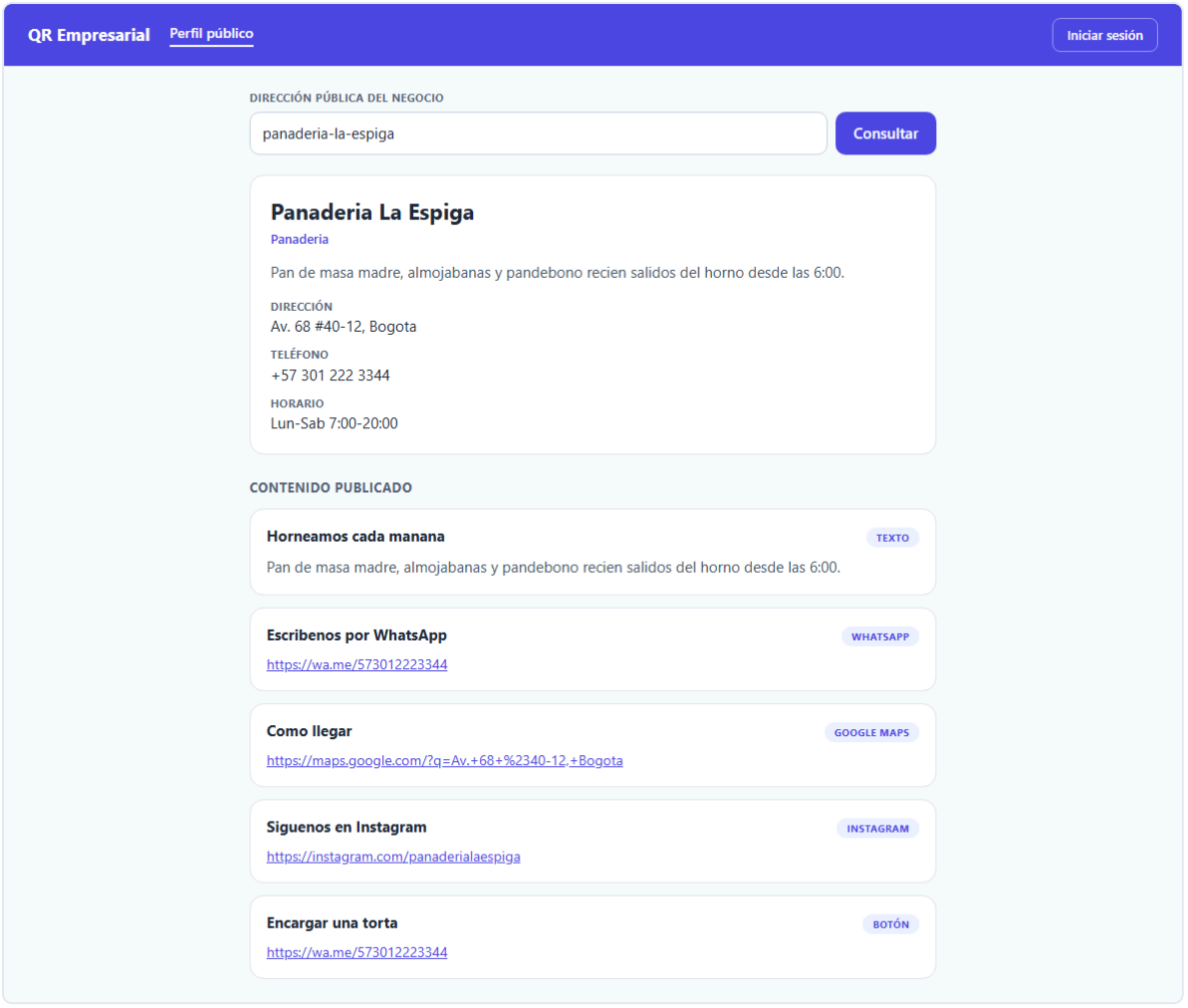


03 · Recuperar la contraseña. Responde lo mismo exista o no el correo, para no revelar qué direcciones están registradas.

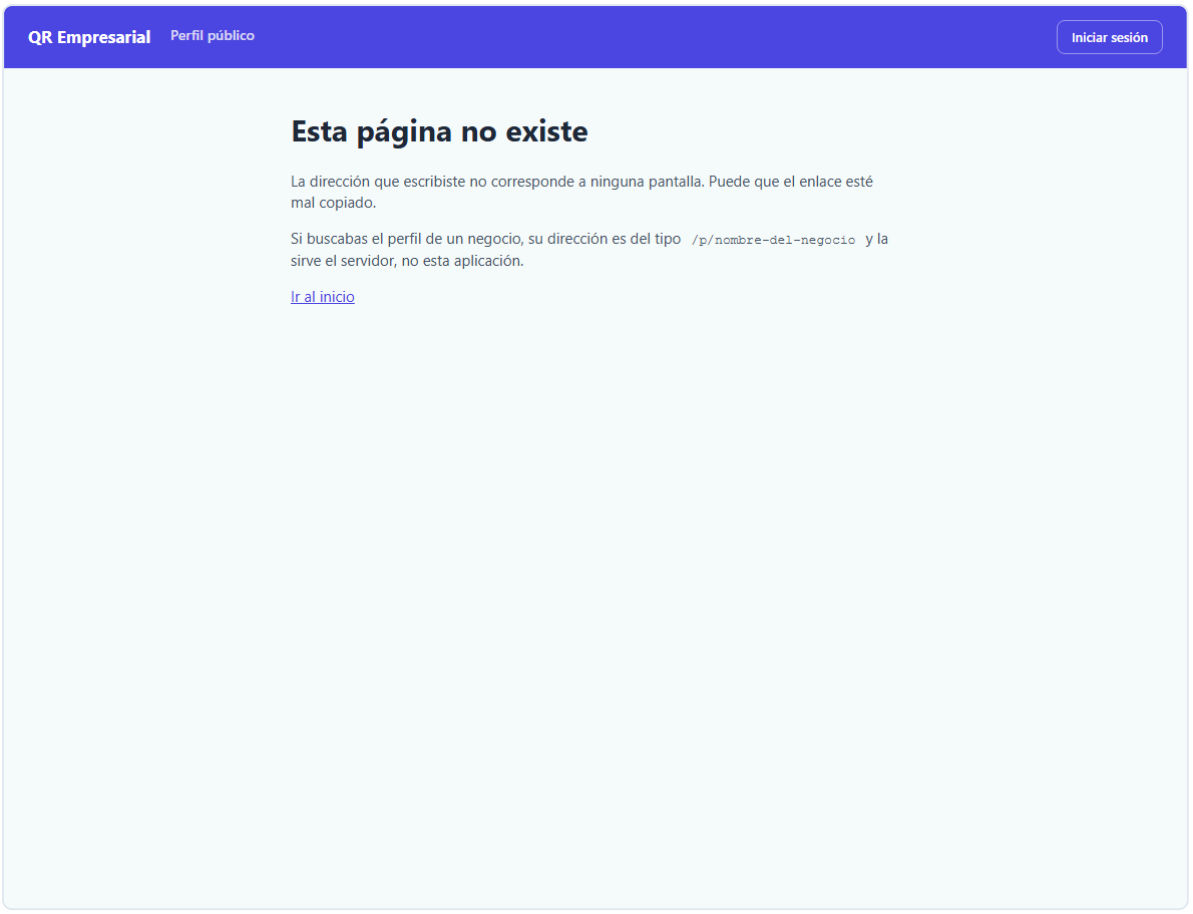


04 · Enlace caducado. Se comprueba antes de enseñar el formulario: pedir una contraseña nueva para luego decir que el enlace no sirve es la peor manera de contarlo.

4.2 Vista pública y errores

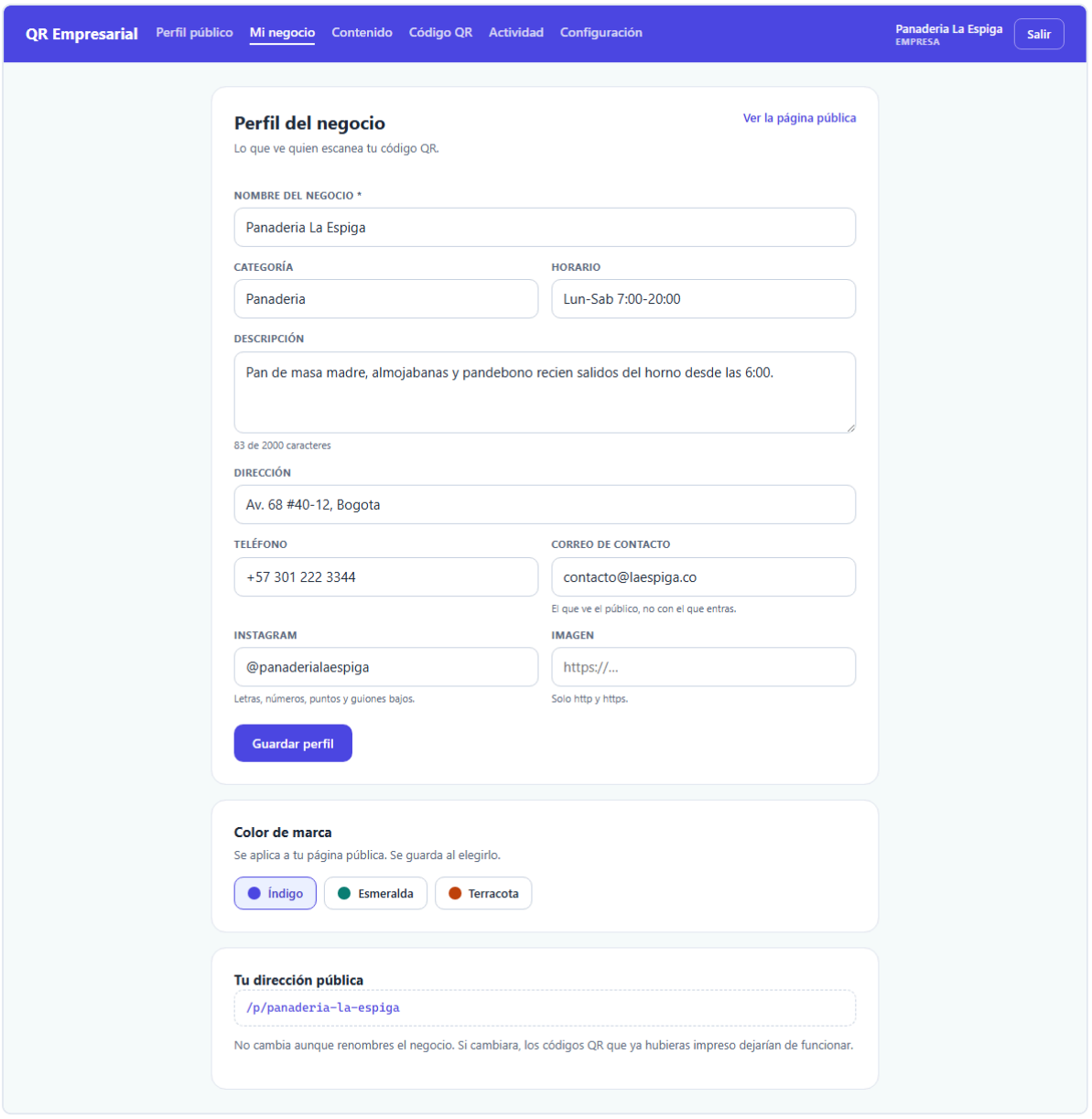


05 · Perfil público. Lo que ve quien escanea el código. No pide sesión. Solo aparecen los bloques activos: el dueño ve seis, el público cinco.

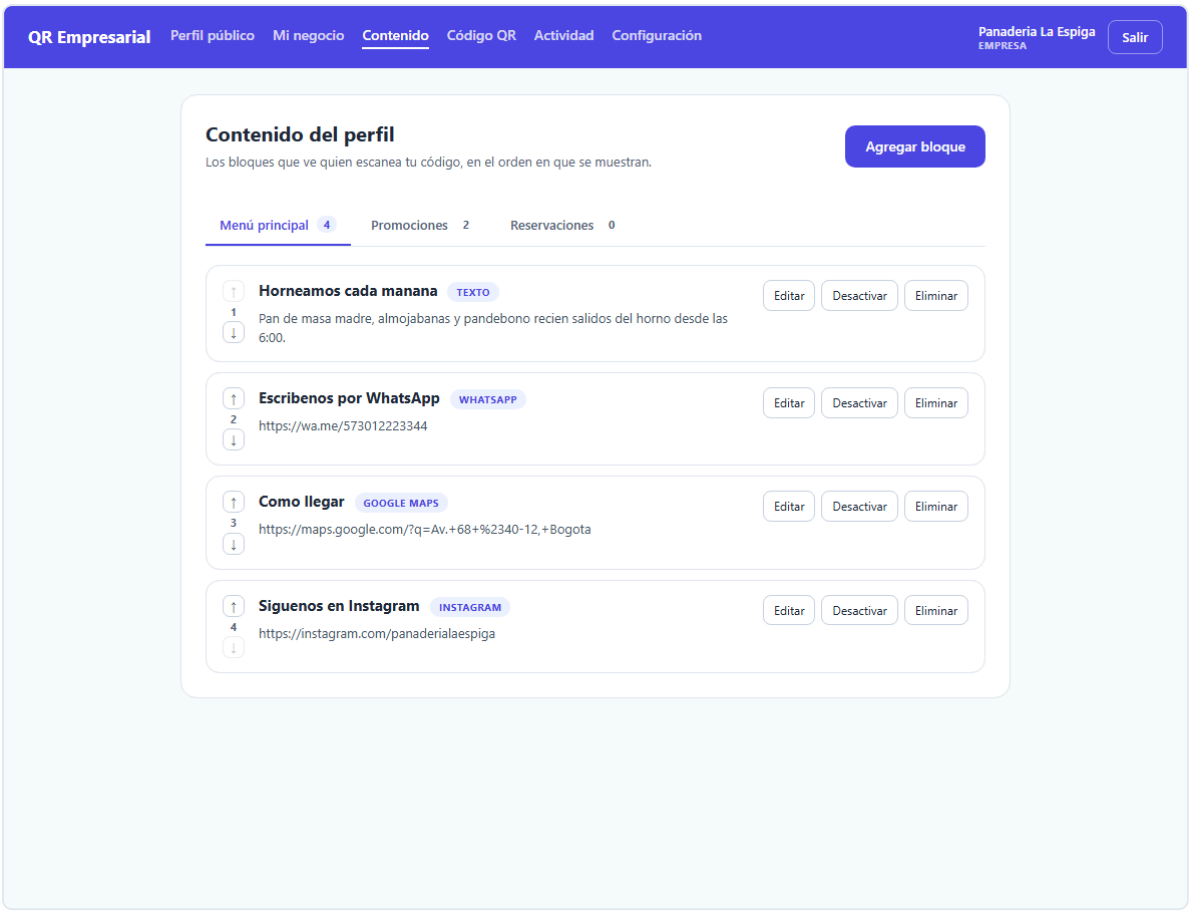


06 · Dirección inexistente. Se dice en lugar de redirigir a la portada en silencio, que haría creer al usuario que llegó donde quería.

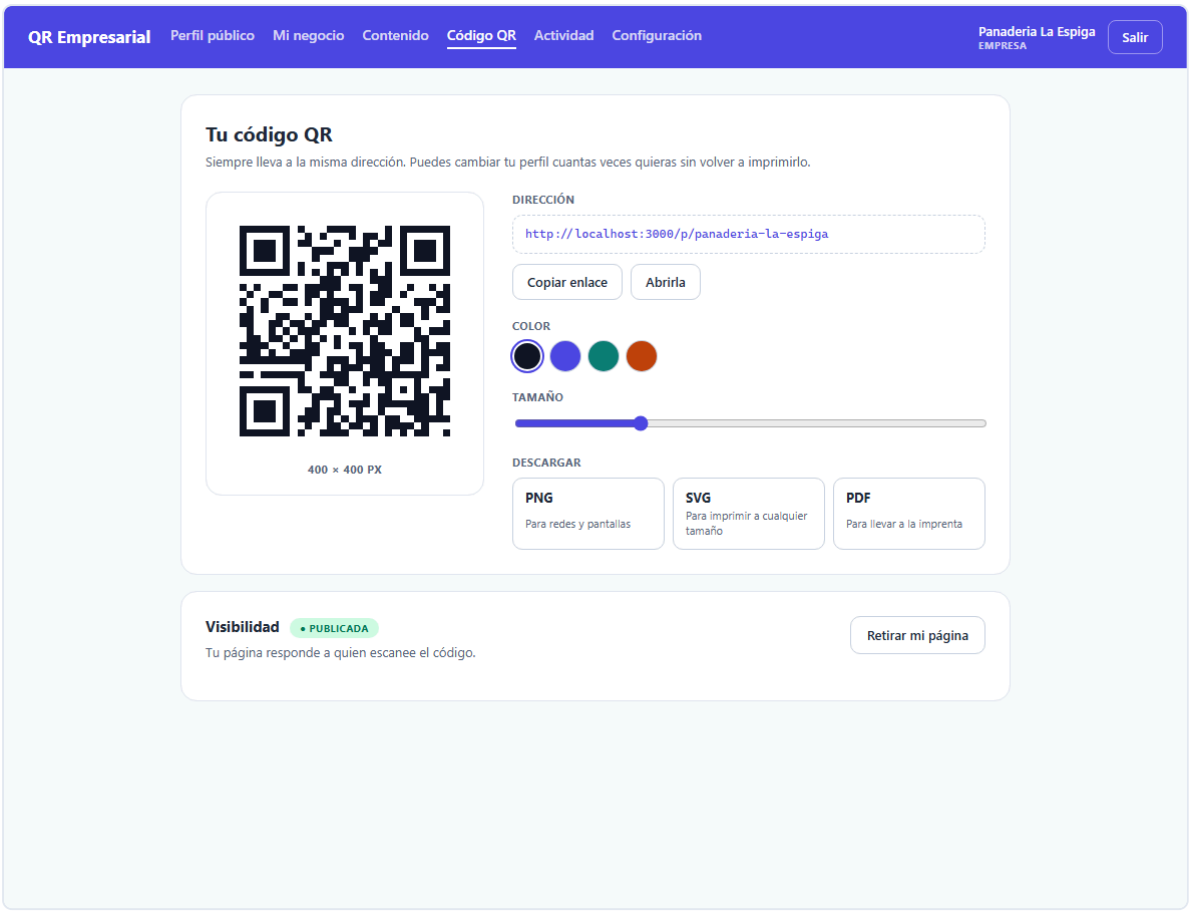
4.3 Panel del negocio



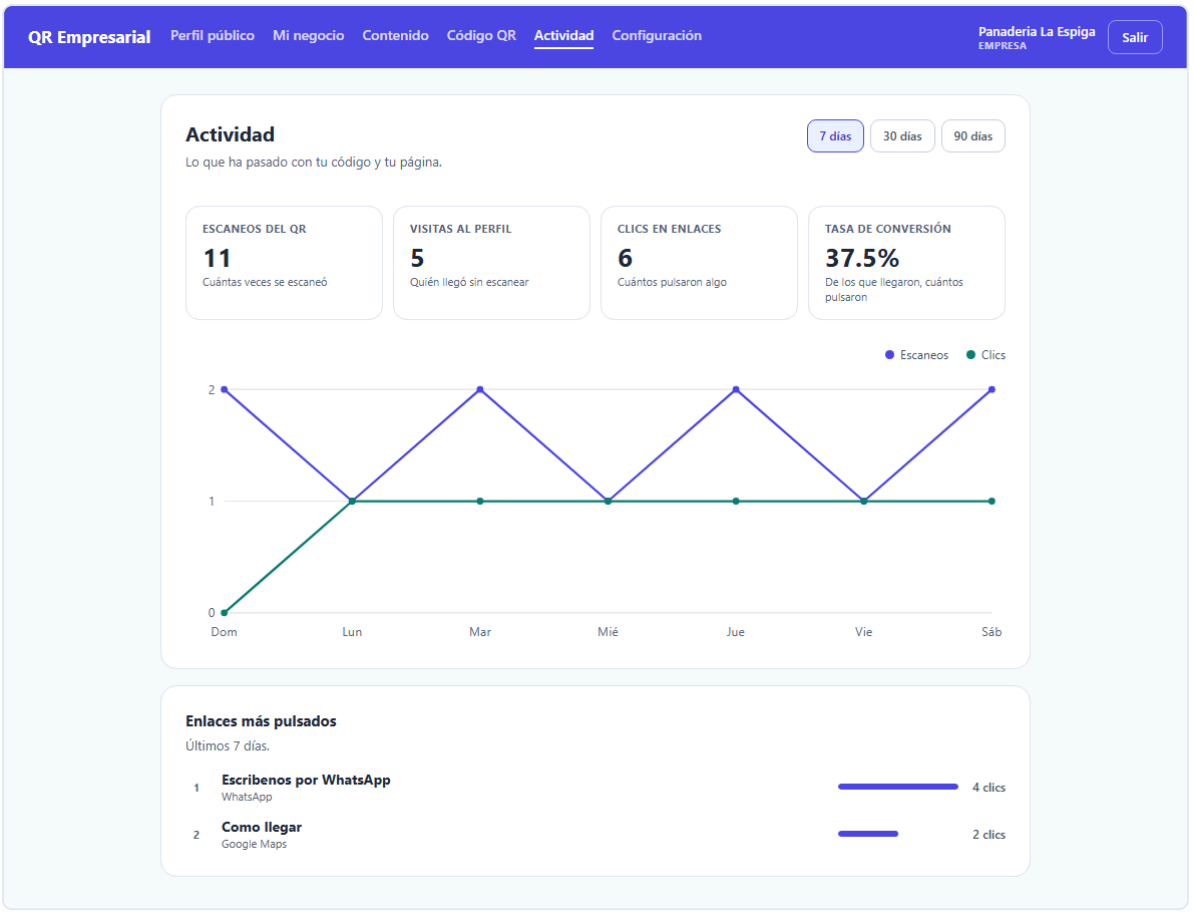
07 · Mi negocio. Los nueve campos del perfil, el color de marca y la dirección pública, que se muestra pero no se edita: si el slug cambiara, los códigos ya impresos dejarían de funcionar.



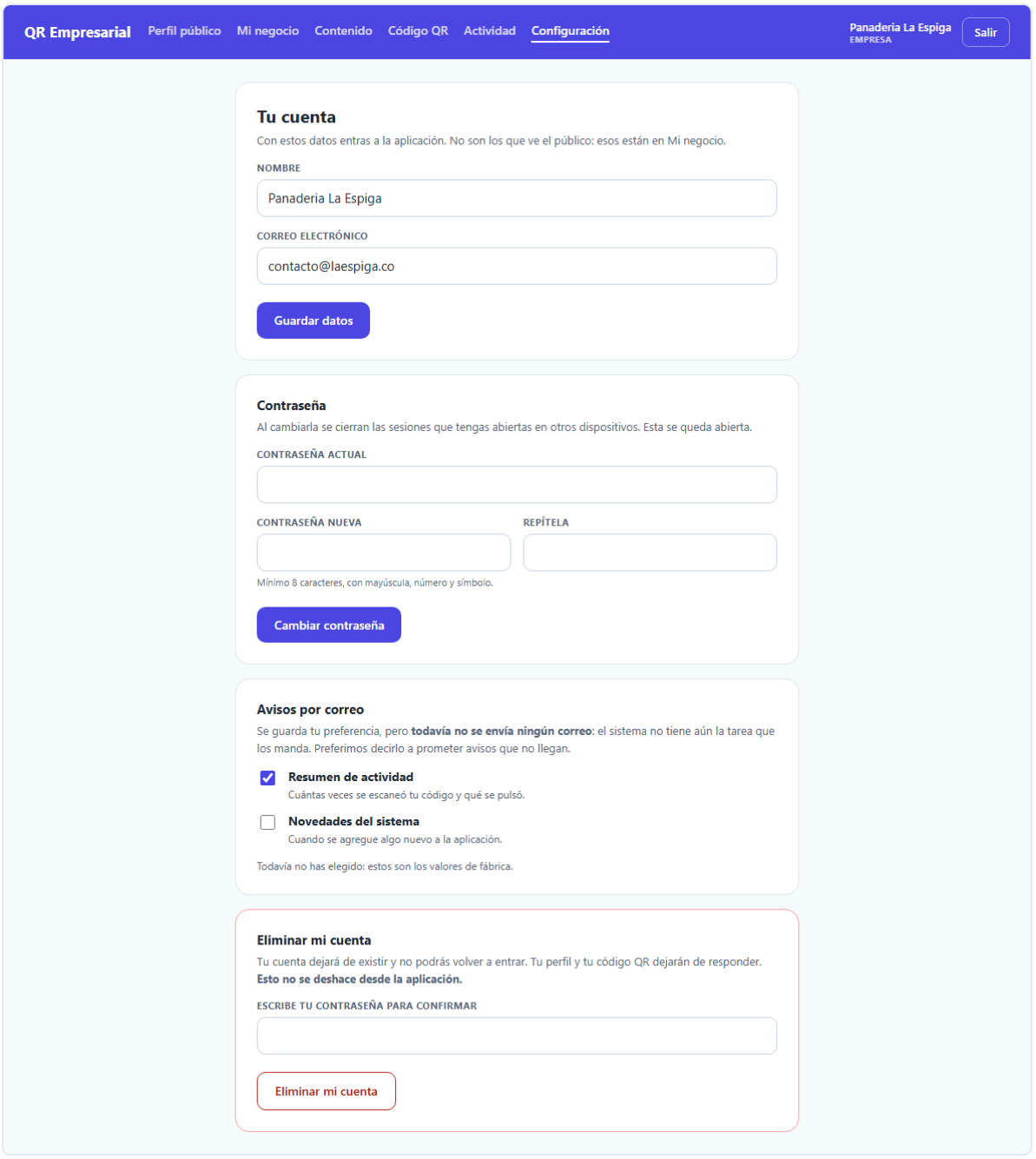
08 · Contenido. Tres secciones con su cuenta de bloques. Se reordena arrastrando y con las flechas: el arrastre de HTML no funciona con el dedo, y sin las flechas un móvil se quedaría sin forma de reordenar.



09 · Código QR. La vista previa la genera el servidor, el mismo endpoint que la descarga: lo que se ve es exactamente el archivo que se va a bajar.

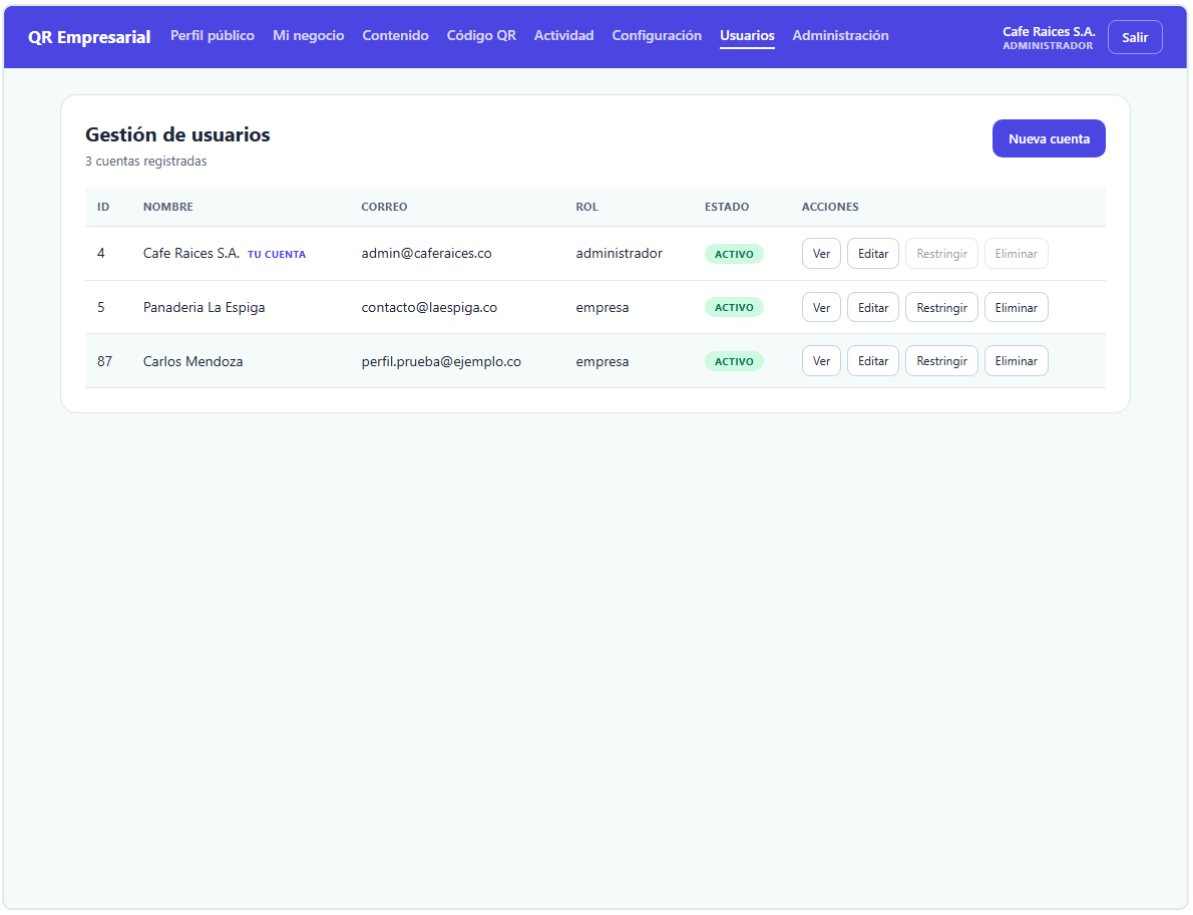


10 · Actividad. Cuatro medidas, la gráfica diaria en SVG y los enlaces más pulsados, en 7, 30 o 90 días —los tres rangos que acepta el servidor—.

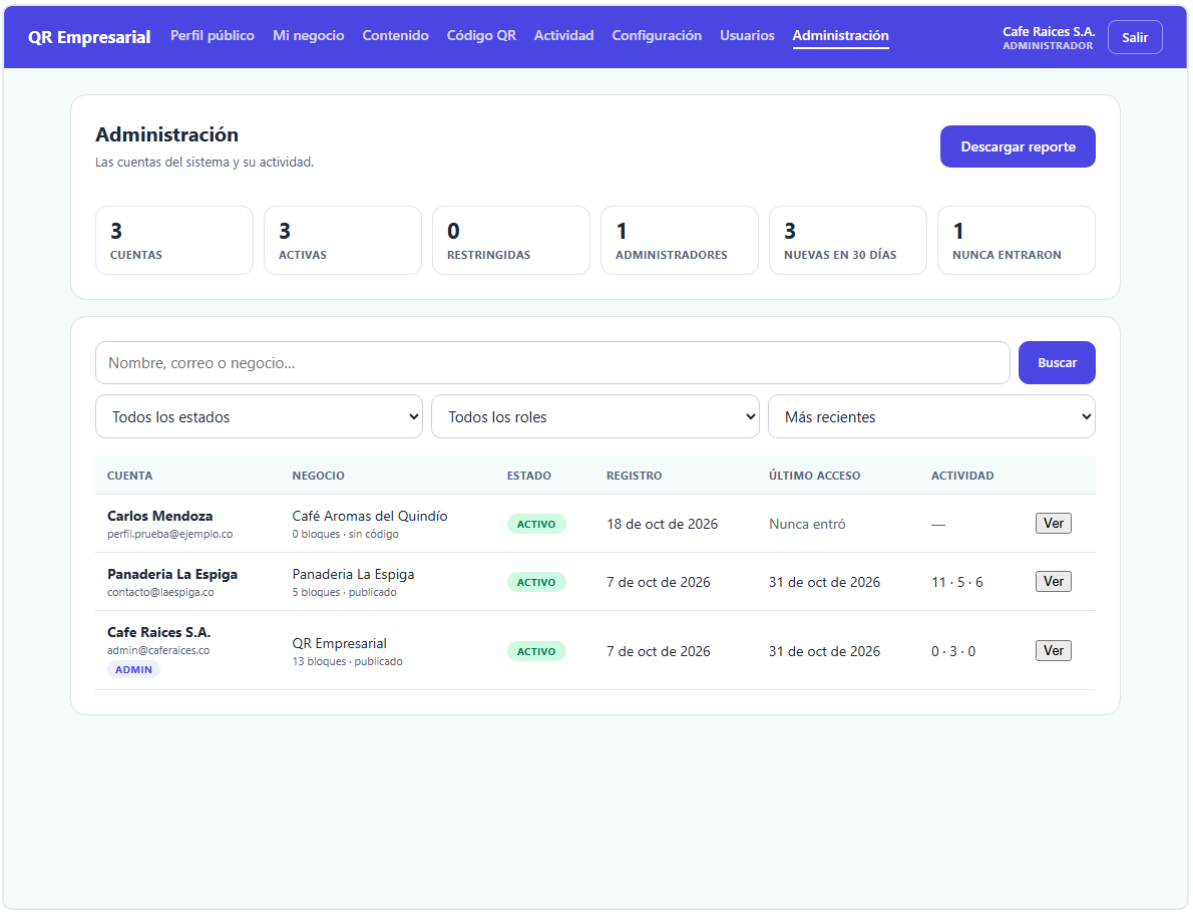


11 · Configuración. Datos, contraseña, avisos y baja. La contraseña solo se pide cuando cambia el correo. Los avisos se guardan pero no se envían, y la pantalla lo dice.

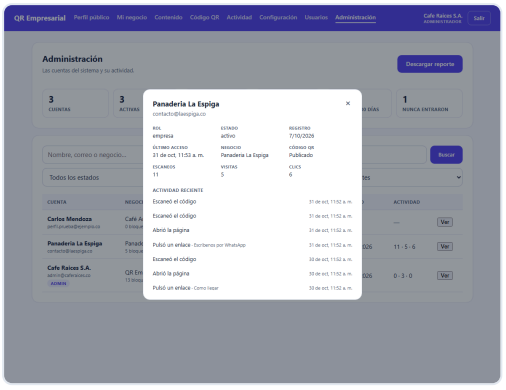
4.4 Administración



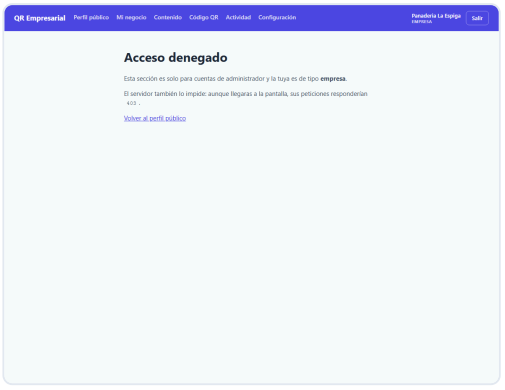
12 · Gestión de usuarios. Los siete endpoints del módulo. Restringir y eliminar están deshabilitados en la propia fila: el servidor lo impide y ofrecer un botón que solo puede fallar es una mala pantalla.



13 · Administración. Seis cifras, filtros con búsqueda, paginación y el reporte en CSV, que respeta los filtros y empieza por BOM para que Excel no rompa las tildes.

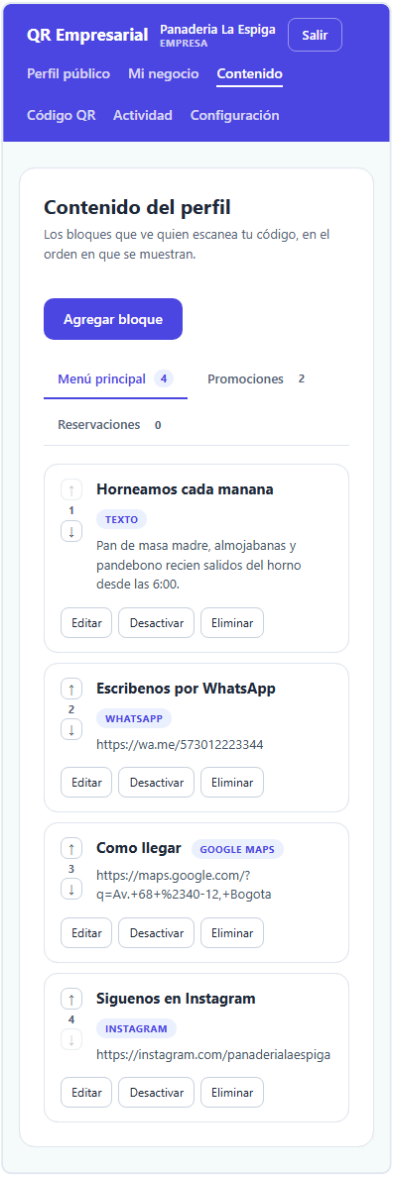


14 · Detalle de una cuenta. La actividad reciente, que la tabla no cabe.



15 · Acceso denegado. Una cuenta de empresa en una ruta de administrador.

4.5 Diseño adaptable



16 · Contenido a 390 px. Sin desplazamiento horizontal: la navegación baja a su propia línea y las acciones de cada bloque se reparten.

5. Repositorio Git

URL	https://github.com/JECAOR/qr-empresarial
Código del frontend	frontend-react/
README	frontend-react/README.md y el de la raíz
Evidencias	frontend-react/evidencias/

5.1 Historial de commits del frontend

Treinta commits sobre `frontend-react/`, del 21 al 31 de octubre de 2026, dentro de un repositorio con **140 commits y 54 etiquetas**. Cada módulo se desarrolló en su propia rama y se integró con `merge --no-ff`, de modo que la rama queda en el historial en lugar de aplanarse. Debajo, los veinticinco últimos; faltan los primeros, por espacio.

```
2026-10-31  feat(frontend-react): icono, manifiesto, service worker     v3.5.0
2026-10-31  feat(frontend-react): un titulo de pestana por pantalla
2026-10-31  feat(frontend-react): aplicar el color de marca (RF-12)
2026-10-31  test(frontend-react): 129 pruebas de extremo a extremo        v3.4.0
2026-10-31  fix(frontend-react): dos medidas por debajo de la WCAG
2026-10-31  fix(frontend-react): el login no devolvia a la pantalla
2026-10-31  feat: el servidor publica el frontend en React                v3.3.0
2026-10-31  docs(evidencias): el documento cuenta las pruebas             v3.2.1
2026-10-31  test(frontend-react): 439 pruebas propias con Vitest          v3.2.0
2026-10-31  docs(evidencias): el documento tambien en Word                v3.1.0
2026-10-31  docs(evidencias): documento de evidencias del frontend        v3.0.1
2026-10-23  feat(frontend-react): el modulo 8, la administracion          v3.0.0
2026-10-23  feat(frontend-react): el modulo 7, la cuenta                  v2.12.0
2026-10-23  feat(frontend-react): el modulo 6, la actividad del perfil    v2.11.0
2026-10-23  feat(frontend-react): el modulo 4, el codigo QR               v2.10.0
2026-10-23  feat(frontend-react): el modulo 3, el contenido del perfil    v2.9.0
2026-10-22  feat(frontend-react): el modulo 2, el perfil del negocio      v2.8.0
2026-10-22  feat(frontend-react): el modulo 1 completo                    v2.7.0
2026-10-22  docs(evidencias): las tres capturas que cierran la Guia 02    v2.6.0
2026-10-22  test(postman): carpeta de la Guia Frontend N.2                v2.5.1
2026-10-22  docs(evidencias): matriz de pruebas y capturas de la Guia 02
2026-10-22  feat(frontend-react): sesion JWT, rutas por rol y usuarios    v2.5.0
2026-10-21  docs(evidencias): la captura de Vite, Guia 01 completa        v2.4.0
2026-10-21  docs(frontend-react): cierra los tres huecos de la Guia 01    v2.3.1
2026-10-21  feat(frontend-react): primera pantalla en React, Guia N.1     v2.3.0
```

Los mensajes siguen **Conventional Commits** y explican el porqué, no solo el qué. El historial completo está en GitHub.

5.2 Cómo ejecutarlo

```
mysql -u root -p --default-character-set=utf8mb4 qr_empresarial < database/seed.sql

cd backend        && npm install && npm run dev    # http://localhost:3000
cd frontend-react && npm install && npm run dev    # http://localhost:5173
```

Las pruebas del frontend no necesitan ninguna de las dos cosas: ni base de datos ni servidor. Las de extremo a extremo compilan la aplicacion y la sirven ellas mismas.

```
cd frontend-react && npm test          # las 474 en jsdom
cd frontend-react && npm run coverage  # con el informe de cobertura

npx playwright install chromium        # solo la primera vez
cd frontend-react && npm run test:e2e  # las 189 en Chromium
```

Cuentas de demostración, las dos con la contraseña `Clave123*`: `admin@caferaices.co` (administrador) y `contacto@laespiga.co` (empresa).

5.3 Defectos encontrados y corregidos

Construir estas pantallas destapó cinco defectos, cuatro en el propio frontend y uno en el backend. Se documentan porque forman parte del proceso:

DEFECTO	CAUSA Y CORRECCIÓN
Un botón se publicaba como texto (backend)	El tipo se decidía comparando el nombre de la categoría con el literal 'Botón', y el cliente mysql de la consola de Windows reescribía los acentos al cargar las migraciones. Se corrigió con SET NAMES utf8mb4 en los dieciséis archivos SQL y comparando por el icono, que es ASCII.
Nombres de columnas en la API (backend)	GET /usuarios devolvía las filas tal como salen de MySQL mientras el resto respondía en camelCase. Nadie lo veía porque el frontend original no llama a esa ruta.
El diálogo se cerraba solo	close() dispara el evento close, que borraba el estado; y StrictMode ejecuta el efecto dos veces. La limpieza sobraba.
El arrastre, que resultó no estar roto	El índice de origen se pasó a useRef creyendo que con estado el arrastre rápido fallaba. Medido después en Chromium —ver 5.6—, no falla: React aplica el cambio de dragstart antes de entregar el drop. La ref se queda porque ahorra un redibujado; la explicación que la acompañaba era falsa y se corrigió.
La barra se quedaba con el nombre viejo	Al cambiar el nombre, la barra seguía mostrando el anterior hasta recargar. AuthContext gana un refrescar().

5.4 Las pruebas automáticas

474 pruebas en 28 archivos, que tardan medio minuto. Se ejecutan con `npm test` dentro de `frontend-react/`. El motor es **Vitest**, que usa la misma configuración que Vite, con **Testing Library** sobre **jsdom**: cada componente se monta de verdad y se consulta como lo haría una persona —por su etiqueta, por su texto, por su papel—, no por el nombre de sus clases CSS.

QUÉ SE PRUEBA	PRUEBAS	LO QUE SE COMPRUEBA
Las quince pantallas	204	El recorrido entero: cargar, escribir, guardar, fallar y reintentar
Los doce componentes	114	Lo que dibujan con cada dato y con qué avisan al pulsarlos
Los once servicios	90	La ruta, el método y el cuerpo de cada llamada, y qué pasa con cada error
El mapa de rutas	28	Que cada dirección lleve a su pantalla y que las protegidas lo estén
El contexto de sesión	11	Que no se fíe del navegador y que tire la sesión si el servidor la rechaza
El guardia de rutas	8	Quién pasa, a dónde se le manda y que no se enseñe nada mientras comprueba
El color de marca y el título	19	Que un tema desconocido no escriba nada en el <html>, que el color de otro negocio no se quede guardado, y que el título lleve el separador del servidor

Cobertura: 99,4 % de las líneas y 96,4 % de las ramas (`npm run coverage`). Ninguna prueba llega a la red: `.env.test` apunta a un servidor que no existe, de modo que una que se escapara fallaría en lugar de tocar la base de datos de desarrollo.

5.5 Cada prueba se falsificó

Una prueba que pasa no demuestra nada si también pasaría con el código roto. Los defectos de la sección anterior se volvieron a meter, uno a uno, para comprobar que su prueba fallaba. Donde la falsificación no tumba todo lo que se esperaría, se dice por qué:

DEFECTO REINTRODUCIDO	QUÉ PASÓ
La fecha de la gráfica leída con new Date()	La etiqueta pasó a decir «30 sep» en lugar de «1 oct»: el mismo día de menos que se vio en pantalla
La limpieza que cerraba el diálogo	Dos pruebas en rojo, incluida la que lo monta en StrictMode
El login que mandaba a todos a /usuarios	Dos pruebas en rojo
Las preferencias que ignoraban al servidor	La nota de «valores de fábrica» no desaparecía: una prueba en rojo
Quitar el refrescar() de la cuenta	Una prueba en rojo
Fiarse del campo success del ejemplo	Diez de las dieciocho de apiClient en rojo
Las dos salidas del login, otra vez	Tres pruebas de extremo a extremo en rojo: el destino recordado aterriza en la portada. Es el defecto de 5.8, y es la prueba de que esas tres discriminan
Invertir los índices del arrastre	Cinco pruebas de extremo a extremo en rojo
Quitar useTemaPropio() de la barra	Cuatro pruebas de jsdom y once de navegador en rojo. Las ocho que siguen pasando son las de la página escaneada, que aplica el color por su cuenta, y las que comprueban que el color de defecto no escriba atributo: son las que no dependen de ese hook
Dejar useTitulo() sin efecto	Diecisiete de las dieciocho de titulos en rojo. La que pasa es la del negocio que no existe, que comprueba justamente que el título NO se toque
Quitar los <link> del icono y el manifiesto, y no registrar el service worker	Cuatro de las quince de pwa en rojo. Las once que pasan solo comprueban que los archivos existan y se sirvan, lo cual sigue siendo cierto: son dos fallos distintos, y un enlace a un archivo que falta es peor que no tenerlo
Devolver la opacidad del 80 % a los enlaces de la barra	Las cuatro auditorías de axe con los otros dos colores en rojo, con color-contrast sobre los enlaces. No hizo falta reintroducirlo: así se encontró

5.6 Dos cosas que estas pruebas no podían demostrar

Las dos salían del mismo sitio: `fireEvent` envuelve cada evento en `act()`, que vacía la cola de React entre uno y el siguiente. Eso esconde todo lo que dependa del momento en que React aplica un cambio. Las dos se resolvieron midiéndolas en un navegador de verdad, y **ninguna acabó donde se creía**.

El arrastre rápido. Se daba por hecho que los tres eventos caen en el mismo tic y que con el índice de origen en estado el arrastre fallaba. No falla: se midió cambiándolo a `useState` y arrastrando en Chromium, despacio y con los tres eventos uno detrás de otro, y reordena igual, porque React trata `dragstart` y `drop` como eventos discretos y aplica el cambio antes de entregar el siguiente. La `ref` se queda porque ahorra un

redibujado de la lista entera por cada arrastre que empieza —eso sí lo mide un `Profiler`—, no porque haga falta. El comentario del componente, que afirmaba una carrera inexistente, se corrigió.

El destino recordado al entrar. Aquí no era una limitación de las pruebas: **era un defecto**. Está en 5.8, porque lo encontró la suite de extremo a extremo y es el motivo de que exista.

> Las dos afirmaciones estaban escritas en el código, en este documento y en el README, y las dos eran falsas. Se corrigen donde estaban, y no solo aquí: una explicación equivocada que sobrevive en un comentario es la que leerá quien toque ese archivo dentro de seis meses.

5.7 El frontend publicado

Desde la versión **v3.3.0** es este el frontend que sirve el servidor: abrir la dirección del proyecto entrega esta aplicación, y los códigos QR llevan a su página `/p/:slug`. El original sigue en el repositorio y se publica cambiando una variable, `FRONTEND=clasico`; sus 396 pruebas de Playwright pasan igual, porque levantan su propio servidor.

Publicarlo obligó a tres cosas que no hacían falta con el original, y que son las que de verdad distinguen servir una aplicación de una sola página:

QUÉ HUBO QUE HACER	POR QUÉ
Reenviar las rutas al index.html	/perfil o /contenido no son archivos: los resuelve el enrutador en el navegador. Sin esto funcionan al navegar y dan 404 al recargar, que es el fallo más fácil de dejar pasar. Solo se reenvían los GET sin extensión: a un POST a una ruta que no existe hay que decirle que no existe, y a un .js que falta, también.
Inyectar las etiquetas de vista previa	WhatsApp, Telegram y Google no ejecutan JavaScript, y esta pantalla se dibuja entera en el navegador: sin inyectarlas en el servidor no habría nada que leer al pegar el enlace. El index.html lleva las mismas marcas que usaba la pantalla del frontend original.
Mover el enlace del correo	React no sirve archivos .html: la recuperación de contraseña apunta ahora a /restablecer. La dirección sale de la configuración y sigue a la variable, porque escribirla en el servicio la habría dejado desfasada al cambiarla.

Y una pantalla nueva, `/p/:slug`, que es la que ve quien escanea: sin buscador —ya llegó a su negocio— y con el registro de escaneos, visitas y clics que hasta ahora hacía el frontend original.

> **Un defecto encontrado al probarlo.** La primera versión guardaba en una `ref` que ya había contado la visita, para que `StrictMode` no la contara dos veces. Sobraba —la bandera de vigencia del efecto ya lo impedía— y además dejaba sin contar la visita al ir de un negocio a otro dentro de la aplicación, porque React Router reutiliza el componente cuando solo cambia el slug. La prueba que lo destapó falla con la `ref` y pasa sin ella.

5.8 Las pruebas de extremo a extremo

189 pruebas en 10 archivos, con **Playwright** sobre Chromium, contra el **paquete compilado**. Se ejecutan con `npm run test:e2e`: compilan la aplicación, la sirven y la apagan al terminar, y no necesitan ni el backend ni MySQL.

Existen por lo que dice 5.6. Las 474 de Vitest corren en jsdom, que no es un navegador: no aplica CSS, no tiene arrastre nativo, no descarga archivos, no tiene `sendBeacon`, no tiene historial ni pestañas, y envuelve cada evento en `act()`. **Aquí no se repite nada que jsdom pueda contestar**: solo lo que necesita un navegador.

ARCHIVO	QUÉ PRUEBA	POR QUÉ NO EN JSDOM
entrar (15)	A dónde se llega al entrar, el historial, Atrás después de salir, la guardia de rol	act() decide quién gana entre dos salidas; no hay historial de navegador
arrastrar (12)	Reordenar arrastrando, despacio y con los tres eventos seguidos; las flechas con ratón y teclado	jsdom no tiene arrastre nativo ni dataTransfer
descargar (14)	PNG, SVG, PDF y el CSV del reporte: nombre y primeros bytes del archivo; la vista previa; el portapapeles	jsdom no descarga nada y createObjectURL es de pega
recargar (17)	Recargar cada pantalla, enlaces directos, direcciones que no existen, la sesión al arrancar en frío	no hay recarga, ni pestañas, ni sessionStorage entre pruebas
escaneada (14)	Escaneos, visitas y clics del RF-11, y que el panel del dueño no cuente nada	sendBeacon no existe en jsdom
movil (19)	Lo que decide el CSS a 375 px: la tabla en tarjetas, la barra que baja, el desbordamiento, el tamaño de lo que se pulsa	Vitest corre con css: false y jsdom no maqueta
accesibilidad (23 x 2)	Auditoría de axe-core en 13 pantallas, 4 estados y los tres colores de marca, el tabulador y el foco visible	la mitad de las reglas de axe miden contraste y tamaños
tema (19)	El color de marca del RF-12: que la barra se pinte con él, que cambie sin recargar, que no parpadee al recargar y que quien escanea vea los colores del negocio	jsdom ve el atributo, pero no que PINTE: Vitest corre con css: false
titulos (18)	El título de la pestaña en cada pantalla, al navegar y al volver atrás	lo que falla es el cambio al navegar, y en jsdom cada pantalla se monta sola
pwa (15)	Que el icono, el manifiesto, robots.txt y el sitemap se sirvan, y que el service worker se registre y abra la aplicación sin conexión	jsdom no pide archivos ni registra service workers

Las dos últimas corren además con el viewport de un teléfono, porque en una pantalla estrecha cambian el contraste de algunos fondos, el orden visual y el tamaño de lo que hay que pulsar.

Encontraron tres defectos en su primera ejecución, y un cuarto al aplicar por fin el color de marca. Es la razón de ponerlas, y la medida de lo que las de jsdom no alcanzaban a ver:

DEFECTO	QUÉ PASABA Y CÓMO SE CORRIGIÓ
El destino recordado al entrar no funcionaba	Quien abría un enlace a /contenido sin sesión entraba y aterrizaba en la portada. La pantalla de login tenía dos salidas: el <Navigate> de «ya tienes sesión» y un navegar(state.desde) después del await. En el navegador ganaba la primera —React agrupa el setUsuario con lo que viene detrás, dibuja una sola vez y desmonta la pantalla—, y el navegar() llegaba sobre un componente desmontado, que React Router ignora en silencio. Ahora hay una sola salida, declarativa.
Las flechas de reordenar medían 22 x 20 px	Por debajo de los 24 x 24 que pide el criterio 2.5.8 de la WCAG 2.2, y son la única forma de reordenar con el dedo, porque el arrastre de HTML no funciona al tacto. Se les puso un mínimo de 32 px.
La etiqueta del rol no tenía contraste	Blanco al 75 % sobre el índigo de la barra da 4,27:1, y hace falta 4,5:1 para texto pequeño. Salía en las nueve pantallas que llevan la barra.
La opacidad no sobrevive a cambiar el fondo	Al aplicar los tres colores de marca, la barra pasó a tener tres fondos. Los enlaces estaban al 80 % de opacidad y la etiqueta del rol al 85 %, medidos los dos sobre el índigo: sobre el esmeralda dan 3,80:1 y 4,09:1, y sobre el terracota 3,85:1 y 4,15:1. La opacidad más baja que servía para los tres era 0,92, con la que la diferencia ya no se nota, así que se quitó: el enlace actual se marca con su subrayado —que no es una diferencia de color, y es lo que pide la WCAG 1.4.1— y el rol por sus 10 px en mayúsculas. La auditoría de axe corre ahora con los tres colores, que es lo que lo encontró.

> El primero es el que importa. La prueba de Vitest no podía verlo, y el comentario que acompañaba al código afirmaba —sin haberlo medido— que en el navegador funcionaba bien. Una suite que no puede fallar donde el código está roto no es una red de seguridad: es una afirmación sobre el código escrita en otro sitio.

La API se simula dentro del navegador, con memoria: reordenar y volver a pedir la lista devuelve el orden nuevo, como haría el servidor. Sin esa memoria, una prueba de reordenar pasaría aunque la pantalla no mandara nada. Y la simulación sale de los servicios del backend, no de lo que le convendría a la prueba: `GET /contenido` devuelve `categoria` como objeto y `GET /publico/:slug` la aplana a una cadena, y simular las dos iguales dejó la pantalla pública en blanco la primera vez. El defecto estaba en la simulación —pero una simulación que no se parece a la API deja pasar exactamente eso en el otro sentido.

5.9 Cuatro cosas que faltaban, y lo que sigue faltando

La versión anterior de este documento declaraba cuatro huecos. Tres eran funcionalidad que el frontend original sí tenía y este no, y se cerraron en la v3.5.0; el cuarto, el título de la pestaña, era un defecto de la aplicación de una sola página. Se dejan escritos porque el valor de la lista era justamente haberlos encontrado.

LO QUE FALTABA	QUÉ ERA Y CÓMO SE RESOLVIÓ
El color de marca no se aplicaba en ninguna parte	Durante cinco versiones la pantalla del perfil dejó elegir entre tres temas, el servidor los guardó y la API los devolvió, pero ninguna pantalla de React los pintaba: se elegía un color y no cambiaba nada. Hasta el texto de la pantalla decía «se aplica a tu página pública», que no era cierto. Era la parte del RF-12 que quedaba pendiente, con el servidor publicando ya este frontend. Ahora lo aplica un módulo que no hace más que poner data-theme en el <html>, y los colores viven en global.css: la barra, los botones y la gráfica se repintan sin recargar la página, que es lo que el original tenía que hacer por generar el QR y las gráficas en un canvas.
Todas las pantallas compartían el título de la pestaña	Decían «QR Empresarial — perfil público» las trece: el login, el panel y la gestión de usuarios. El original tenía doce títulos, uno por archivo. Ahora lo pone un hook en cada pantalla, con el mismo separador que usa el servidor al armar las etiquetas de /p/{slug}; en esa pantalla el título es el nombre del negocio, y mientras no se sabe no se toca, para no borrar el que el servidor escribió para los robots.
No había icono de pestaña	El <head> no tenía ninguna etiqueta de icono, así que el navegador solo buscaba /favicon.ico —que no existe— y la pestaña salía sin icono. Los archivos estaban publicados en dist/assets desde el principio y nada los enlazaba.
No había manifiesto, service worker ni robots.txt	El original tenía los cuatro. Con ellos la aplicación se puede instalar y abre sin conexión una vez visitada. El service worker va a red primero y caché de respaldo, y no guarda nada de /api/: son datos de una cuenta y en un equipo compartido la caché devolvería lo de quien usó el navegador antes. El robots.txt importa más aquí que en el original, porque las direcciones de React no son archivos: un robot que pida /contenido recibe un 200, no un 404, así que sin reglas indexaría el panel entero.

Lo que sigue faltando, y no se ha resuelto:

QUÉ FALTA	DETALLE
Safari de verdad	El proyecto Móvil usa Chromium con el viewport y el user agent de un iPhone, porque este entorno no pudo instalar un WebKit real. Lo que se mide —media queries, cajas, contraste— no depende del motor; un fallo propio de WebKit sí se escaparía.
El doble paso de StrictMode, en el navegador	Solo ocurre en desarrollo, y las de extremo a extremo corren contra el paquete compilado. Ese caso lo cubre Vitest.
La vista pública no separa las secciones	La API devuelve el contenido agrupado por sección —porSeccion— y la pantalla lo dibuja en una sola lista plana. El panel sí organiza el contenido en tres pestañas (Menú principal, Promociones, Reservaciones), así que quien publica las usa y quien visita no las ve separadas.
No hay pantalla de menú	El original tenía una página de menú digital propia, con su enlace desde el perfil público, que es la parte del RF-05 que la separa del resto del contenido. En React no existe esa ruta. Es consecuencia de lo anterior, pero es una pantalla entera menos.

> Las dos últimas se encontraron comparando pantalla por pantalla los dos frontends, al preguntar si el de React estaba terminado. No lo estaba: una comparación de lo que cada uno sabe hacer dice más que la suma de las pruebas que pasan.